Katmai Bear Pipeline — Behavior & Identity User Guide

Detect, describe, and **identify by name** every brown bear catching salmon in a Brooks Falls / Brooks River video.

What this gives you

Given a video like this:

(12-second clip of a single bear catching salmon at Brooks Falls)

the pipeline produces a side-by-side demo video where the right panel shows:

```
● Plunger
  [CATCHING] The bear is clamping down on a salmon, with the
  fish visible in its jaws as it emerges from the water.
```

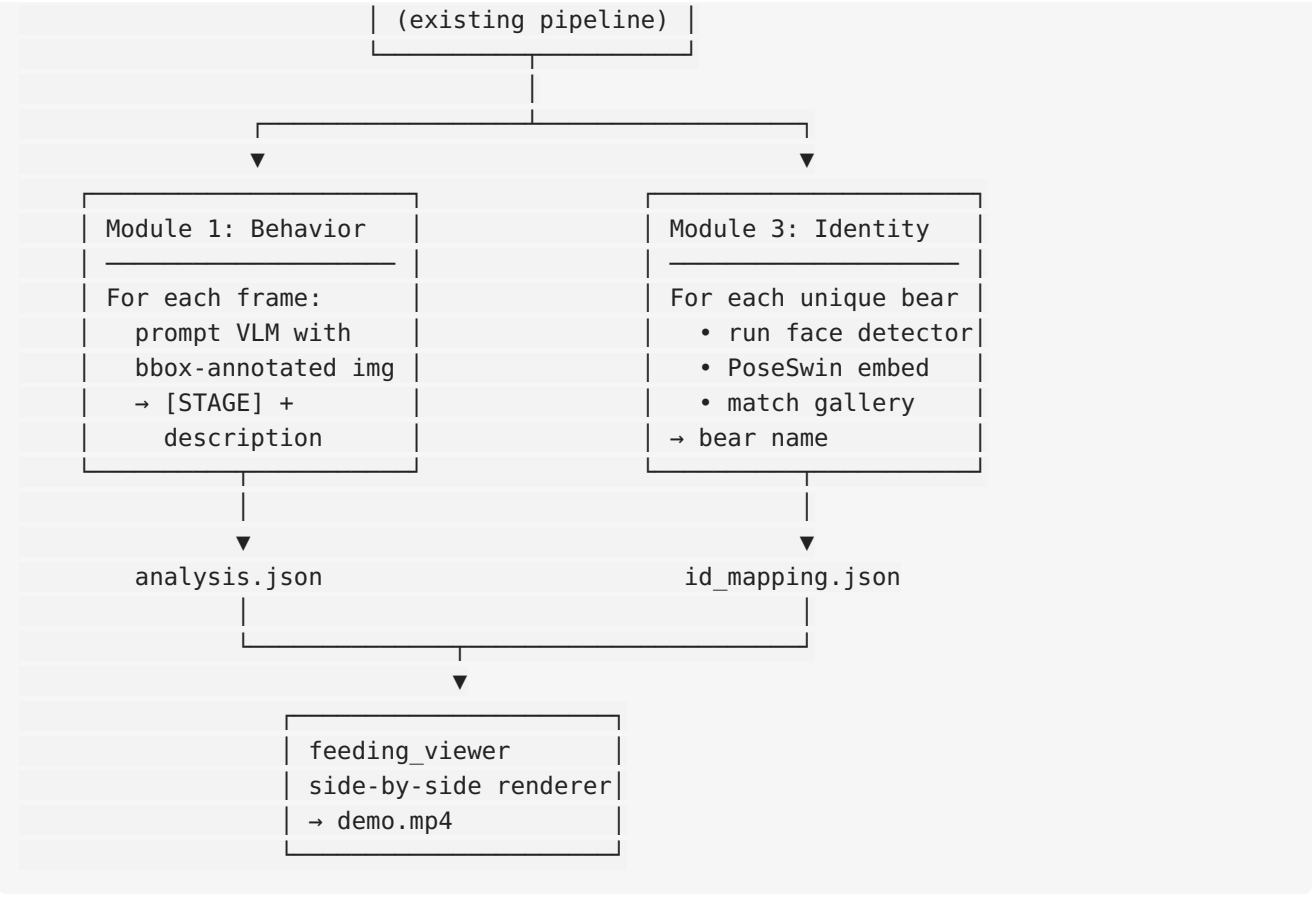
— that is, **per-frame feeding-stage classification** (`WAITING` , `LUNGING` , `CATCHING` , `EATING` , `MISSED`) for each bear, combined with a **persistent cross-video identity** (e.g., `Plunger` , `Bony_Butt`) so the same physical bear keeps the same name across many videos.

The three modules

This guide covers three independent modules. You can run any subset:

#	Module	Status	Purpose
1	**Behavior classification** (`src/behavior/analyze_feeding.py`)	✅ Production	Classifies each bear's feeding stage every N seconds via a vision-language model (default: Molmo2-8B, **pluggable**).
2	**Pixel-based eating detection** (`src/behavior/pixel_eating_detector.py`)	⚠️ Research artifact	Lightweight CPU-only detector using HSV color analysis + bbox aspect ratio. **Did not work** on our side-view footage; kept as documented reference. See `pixel_detection_attempts_report.md` .
3	**Cross-video bear identity** (`src/identity/`)	✅ Production	Identifies which named bear is which using PoseSwin face embeddings; matches against a persistent gallery so the same bear keeps the same name across videos.

Diagram of the full pipeline (modules 1 and 3 are typically run together):



Quick start

Prerequisites

- NVIDIA GPU with ⩾ 16 GB VRAM (Module 1 with default Molmo2 backend; less if you use a cloud API instead — see § "Bring Your Own Backend")
- Python 3.10
- The repo's existing `venv/` (already set up; uses PyTorch 2.6, transformers 4.57.6)

One-time setup for Module 3 (identity)

The PoseSwin model + gallery image data come from a 30 GB public dataset:

```
cd external/BrownBear_ReID

# 1. Download the Public_release.zip (30 GB, ~30 min over good link)
curl -L -o Public_release.zip "https://zenodo.org/records/17822054/files/Public_release.zip?downloa

# 2. Extract just the checkpoints (4.2 GB)
unzip -q Public_release.zip "Public_release/checkpoints/*"

# 3. Build the named gallery (one-time, ~7 min on dual 2080 Ti)
cd ../..
WANDB_MODE=disabled venv/bin/python3 -m src.identity.build_named_gallery \
```

```
    --image-root data/identity/gallery_images \
    --output     data/identity/named_bear_gallery.json
```

If you only want Module 1 (behavior), you can skip all of this.

Run end-to-end on one video

```
cd /path/to/katmai-cv-pipeline

VIDEO=feed/data_video/your_clip.mp4

# Step 1 — Behavior classification (per-frame [STAGE] + descriptions)
WANDB_MODE=disabled venv/bin/python3 -m src.behavior.analyze_feeding \
    --video    "$VIDEO" \
    --interval 0.25

# Step 2 — Bear identification (assign cross-video names)
WANDB_MODE=disabled venv/bin/python3 -m src.identity.identify_bears \
    --video     "$VIDEO" \
    --analysis  predictions/$(basename ${VIDEO%.*})_feeding_analysis/analysis.json \
    --gallery   data/identity/named_bear_gallery.json \
    --threshold 0.45

# Step 3 — Render the demo video with identity + behavior
WANDB_MODE=disabled venv/bin/python3 -m src.behavior.feeding_viewer \
    --video       "$VIDEO" \
    --analysis    predictions/$(basename ${VIDEO%.*})_feeding_analysis/analysis.json \
    --id-mapping  predictions/$(basename ${VIDEO%.*})_feeding_analysis/id_mapping.json
```

The demo video will be at `predictions/<video_stem>_feeding_analysis/<video_stem>_feeding_demo.mp4`.

Module 1: Behavior classification

What it does

Samples one frame every `--interval` seconds. For each sample, runs YOLO + ByteTrack to get bear bboxes, then sends the annotated frame to a vision-language model (VLM) which returns:

```
Bear 1: [WAITING] Standing at the edge of the waterfall, scanning the water below for salmon.
Bear 2: [CATCHING] The bear has its mouth clamped around a salmon...
```

Outputs an `analysis.json` with per-bear behavior at every sample timestamp, plus a 2-4 sentence summary of the whole clip.

Tuning

Flag	Default	Notes
`--interval`	0.5	Seconds between samples. **0.25 catches more events but costs 2× GPU**.
`--conf`	0.25	YOLO bear detection confidence threshold
`--iou`	0.7	NMS IoU. Lower (0.45) helps with crowded scenes.
`--dedupe-threshold`	0.7	Avoid re-emitting identical descriptions on adjacent samples

Bring Your Own VLM Backend

By default Module 1 uses **Molmo2-8B** locally (~22 GB VRAM, ~5–8 s/frame). You can swap in any of these without touching the rest of the pipeline:

Backend	CLI flag	Setup	Pros / Cons
Molmo2-8B (default)	`--backend molmo2`	none — already installed	Local; no API cost; free; Apache 2.0 weights. Needs 22 GB VRAM.
OpenAI GPT-4o	`--backend openai`	`pip install openai` ; `export OPENAI_API_KEY=...`	Best raw quality; no local GPU. ~$0.01–0.03/frame; uploads frames to OpenAI.
Anthropic Claude	`--backend anthropic`	`pip install anthropic` ; `export ANTHROPIC_API_KEY=...`	Excellent at structured output. Per-frame API cost.
Google Gemini	`--backend gemini`	`pip install google-generativeai` ; `export GOOGLE_API_KEY=...`	Cheap; students get free quota. Quality varies.

Examples:

```
 # Default (local Molmo2)
venv/bin/python3 -m src.behavior.analyze_feeding --video clip.mp4

# OpenAI GPT-4o (mini)
venv/bin/python3 -m src.behavior.analyze_feeding --video clip.mp4 \
    --backend openai

# Claude Sonnet 4.6
venv/bin/python3 -m src.behavior.analyze_feeding --video clip.mp4 \
    --backend anthropic --vision-model claude-sonnet-4-6

# Specific OpenAI model
venv/bin/python3 -m src.behavior.analyze_feeding --video clip.mp4 \
    --backend openai --vision-model gpt-4o
```

Adding a brand-new backend (e.g. another open-weights VLM)

1. Subclass `BaseBehaviorBackend` from `src/behavior/backends/base.py` :

```python # src/behavior/backends/my_model.py from .base import BaseBehaviorBackend
```

class MyModelBackend(BaseBehaviorBackend): name = "my_model"

```
    def __init__(self, model_name="org/my-vlm-7b", **kw):
        # load your model here
        ...

    def analyze_frame(self, image_pil, prompt: str) -> str:
        # return raw text — pipeline parses "Bear N: [STAGE] description" lines
        ...

    def summarize_video(self, timeline_text, reference_image_pil=None) -> str:
        ...
```

```

1. Register it in `src/behavior/backends/__init__.py`:

`python if name == "my_model": from .my_model import MyModelBackend return MyModelBackend(**kwargs)`

1. Use it: `--backend my_model`

That's it. The backend never sees the rest of the pipeline.

---

## Module 2: Pixel-based eating detection (research artifact)

This was an experiment to detect bear eating using **pure pixel analysis** — HSV thresholding for salmon-flesh colors (pink/red/white) plus bbox aspect-ratio posture heuristics. **It did not work** on our typical Brooks Falls side-view footage:

- Salmon at Brooks Falls in early/mid summer are silvery, not pink-spawning red
- The fish is mostly inside the bear's mouth (< 2% of bbox pixels)
- Bear fur is brown — the "warm color" signal is dominated by the bear, not the salmon

We measured a **−0.013 separation** between `CATCHING` and `WAITING` frames (essentially noise; ROC AUC ≈ 0.5). The full analysis is in `pixel_detection_attempts_report.md`.

The code is kept as a reference + as a potential **fast pre-filter** for a future hybrid pipeline (pixel detector rejects obviously-non-eating frames → VLM only inspects candidates → 3× GPU cost reduction). Run it like:

```
venv/bin/python3 -m src.behavior.pixel_eating_detector \
    --video   clip.mp4 \
    --analysis predictions/clip_feeding_analysis/analysis.json \
    --render
```

---
```

Module 3: Cross-video bear identity

What it does

For each unique bear (ByteTrack ID) in a video, picks the highest-confidence frames, runs the **bear face detector** (Faster-RCNN ported from mmdetection to torchvision — see `bear_identity_pipeline.md` for the conversion details), feeds the head crop to **PoseSwin** (Rosenberg et al., Current Biology 2026) to get a 512-dim face embedding, and matches against a **persistent gallery** of known bears.

The same physical bear gets the **same name** across all videos because the gallery is a single JSON file shared across runs.

Bring Your Own Bear Photos

The default gallery has 98 bears from the **McNeil River** training set (Plunger, Bony_Butt, Simba, Aardvark, Hotlips, ...). These names are useful for cross-video persistence but **don't correspond to the named Brooks Falls bears** (Otis 480, Grazer 128, etc.) that the public knows.

To build a Brooks Falls gallery — or any custom gallery — collect a few photos of each bear you want to recognize:

```
my_bears/
  Otis_480/
    photo1.jpg
    photo2.jpg          # 5–15 photos per bear is plenty
    ...
  Grazer_128/
    ...
  747/
    ...
```

Then add them to the gallery:

```
 # A. Add to existing gallery (merge with old entries)
venv/bin/python3 -m src.identity.add_to_gallery \
    --image-root my_bears \
    --gallery    data/identity/named_bear_gallery.json

# B. Or build a fresh Brooks Falls-only gallery
venv/bin/python3 -m src.identity.add_to_gallery \
    --image-root my_bears \
    --gallery    data/identity/brooks_falls_gallery.json \
    --replace

# C. If your photos are already cropped head shots, skip the face detector
venv/bin/python3 -m src.identity.add_to_gallery \
    --image-root my_bears \
    --no-face-detector
```

After that, point `identify_bears.py` at the new gallery:

```
venv/bin/python3 -m src.identity.identify_bears \
    --video    clip.mp4 \
    --analysis predictions/clip_feeding_analysis/analysis.json \
    --gallery  data/identity/brooks_falls_gallery.json
```

Tuning identity matching

Flag	Default	Notes
`--threshold`	0.45	Cosine similarity required to call it a match. Lower = more matches but more false positives.
`--top-k`	10	How many highest-confidence frames to embed per bear (more = more robust)
`--face-score-threshold`	0.3	Faster-RCNN face score required to use that crop (lower = more crops, possibly worse)
`--no-face-detector`	off	Skip the face detector; use heuristic head crop. Faster but less accurate.

Output reference

`analysis.json` (Module 1)

```
{
  "video": "/path/to/clip.mp4",
  "fps": 60.0,
  "interval_sec": 0.25,
  "backend": "molmo2",
  "summary": "The video shows ...",  // 2–4 sentence narrative
  "entries": [
    {
      "timestamp_sec": 1.25,
      "frame_idx": 75,
      "bears": {
        "1": {
          "bbox": [12, 473, 524, 892],
          "conf": 0.96,
          "behavior": "[CATCHING] The bear is clamping ..."
        }
      }
    },
    ...
  ]
}
```

`id_mapping.json` (Module 3)

```
{
  "video": "/path/to/clip.mp4",
  "gallery_path": "data/identity/named_bear_gallery.json",
```

```
  "threshold": 0.45,
  "mapping": {
    "1": {
      "name": "Plunger",
      "similarity": 0.851,
      "is_new": false,
      "n_shots": 10,
      "n_face_crops": 2,
      "n_heuristic_crops": 8,
      "max_conf": 0.97
    }
  }
}
```

`bear_gallery.json` (persistent across runs)

```
{
  "next_anon_idx": 0,
  "entries": [
    {
      "name": "Plunger",
      "embeddings": [[0.123, -0.045, ...]],   // 512-d, L2-normalized
      "n_observations": 15
    },
    ...
  ]
}
```

Code map

```
src/
├── behavior/
│   ├── analyze_feeding.py      ← Module 1 main: YOLO+ByteTrack → backend → analysis.json
│   ├── feeding_viewer.py       ← side-by-side renderer (reads analysis.json + id_mapping.json)
│   ├── pixel_eating_detector.py← Module 2 (research artifact, see notes above)
│   └── backends/
│       ├── base.py             ← Abstract BehaviorBackend interface
│       ├── molmo2.py           ← default local Molmo2-8B
│       ├── openai_gpt4o.py     ← OpenAI API
│       ├── anthropic_claude.py ← Anthropic Claude API
│       └── gemini.py           ← Google Gemini API
│
└── identity/
    ├── poseswin_identifier.py  ← PoseSwin model wrapper + Gallery class
    ├── face_detector.py        ← Faster-RCNN bear-head detector (mmdet → torchvision)
    ├── identify_bears.py       ← Module 3 main: analysis.json → id_mapping.json
    ├── build_named_gallery.py  ← Build initial gallery from PoseSwin training data
    └── add_to_gallery.py       ← Add user-supplied bear photos (BYO)

data/identity/
├── named_bear_gallery.json     ← Persistent gallery (98 named bears from PoseSwin)
```

```
└── gallery_images/             ← Reference photos used to build the gallery
    ├── Plunger/
    │   ├── *.JPG
    └── └── ...

predictions/<video_stem>_feeding_analysis/
├── analysis.json               ← Module 1 output
├── id_mapping.json             ← Module 3 output
└── <video_stem>_feeding_demo.mp4  ← Final side-by-side demo
```

Related documents

- **`bear_identity_pipeline.md`** — deep technical doc on Module 3 (PoseSwin + Faster-RCNN integration, mmdet→torchvision weight conversion details, empirical results)
- **`eating_detection_design.md`** — design discussion of consumer-grade vs cloud paths for eating detection
- **`pixel_detection_attempts_report.md`** — full negative-result report of Module 2 (5 methods tried, why each failed)
- **`models_and_tools_report.md`** — Alex's 8-question model report (English version)
- **`PoseSwin_smoketest.pptx`** — original smoke-test deck

Troubleshooting

"AttributeError: 'IterableSimpleNamespace' object has no attribute 'fuse_score'" Add `fuse_score: True` to `configs/trackers/bytetrack.yaml`. This is required by ultralytics ⩾ 8.x.

"Unexpected keyword argument image_use_col_tokens" (Molmo2) You have transformers ⩾ 5.x. Pin to 4.57.6: `pip install transformers==4.57.6`.

"Using or_mask_function arguments require torch>=2.6" Upgrade: `pip install torch==2.6.0+cu124 -f https://download.pytorch.org/whl/torch_stable.html`.

Module 3 says all bears get a new identity (no matches) Lower `--threshold` (e.g. 0.4 or 0.35), or add more reference photos via `add_to_gallery.py`, or check that `data/identity/named_bear_gallery.json` actually exists.

OOM during `generate_summary` The reference frame is downsized to 512 px max; if you still OOM, reduce `--interval` so fewer frames are kept in `raw_frames` memory.

OOM in face detector Pass `--no-face-detector` to `identify_bears.py` to fall back to heuristic head crop (slightly less accurate but no Faster-RCNN GPU usage).

License

- Pipeline code: see project root `LICENSE`
- Molmo2-8B weights: Apache 2.0

- PoseSwin model + dataset (BrownBear_ReID): **CC BY-NC 4.0** (non-commercial)
- YOLOv8 (Ultralytics): AGPL 3.0
- OpenAI / Anthropic / Google APIs: subject to vendor terms

If your downstream use is commercial, the PoseSwin license is the binding constraint — talk to the EPFL Mathis Lab or use a permissively-licensed Re-ID method.

Citation

If you publish using this pipeline, please cite:

```
@article{rosenberg2026poseswin,
  title={Individual identification of brown bears using pose-aware metric learning},
  author={Rosenberg, Beth and Zhou, Mu and Wolf, Nathan and Mathis, Mackenzie W
          and Harris, Bradley P and Mathis, Alexander},
  journal={Current Biology},
  year={2026},
  doi={...}
}

@article{molmo2024,
  title={Molmo and PixMo: Open Weights and Open Data for State-of-the-Art Multimodal Models},
  author={Allen Institute for AI},
  year={2024}
}

@inproceedings{liu2021swin,
  title={Swin Transformer: Hierarchical Vision Transformer using Shifted Windows},
  author={Liu, Ze and others},
  booktitle={ICCV},
  year={2021}
}
```